

Internal verification of assessment decisions – BTEC Higher Nationals (RQF)

INTERNAL VERIFICATION – ASSESSMENT DECISIONS (Single Student)					
Programme Title:					
Unit Number and Title:					
Assessor Name:			Internal Verifier Name:		
Assignment title:					
Name of student	Submission Type (First, Resubmission, Retake)	Grade the Assessor has awarded.	Assessment Decision Accurate (Y/N) (Resubmission and retake must be capped at a Pass)	List the learning outcomes and grading criteria where inaccurate decisions have been made	State why the assessment decision is inaccurate. <i>*If an inaccurate decision is recorded the Internal Verifier must recommend actions detailing the issues to be addressed. The Assessor and the Internal Verifier must then confirm that the action has been undertaken before assessment decisions are issued to the student.</i>
INTERNAL VERIFIER CHECKLIST					Y/N
Has the student and the Assessor confirmed the authenticity of the evidence?					
Is there evidence of collusion or plagiarism? The work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. The evidence submitted for this assignment is student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work.					

Is the assessor feedback to the student appropriate and constructive to each student? Provide comments below.

- Points out strengths and areas for improvement.
- Linked to relevant learning outcomes and assessment criteria.
- Clear as to why the student did not achieve higher grades.
- Identify opportunities for improved performance in future assignments.

GENERAL COMMENTS

Any actions required must be reviewed across the whole cohort.

Action Required	Target Date for Completion	Date Action Completed

I confirm that the assessment decisions are accurate, there is no evidence of assessment malpractice and any action points have been addressed and completed in respect of the whole cohort. I declare the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. The evidence submitted for this assignment is student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work.

Internal Verifier signature		Date	
Assessor signature		Date	

Higher Nationals - Summative Assignment Feedback Form

Student Name/ID	MN. Sihanass / 1030371		
Unit Title	Data Structures & Algorithms		
Assignment	Assignment 01	Assessor	
Submission Date		Date Received 1st	
Re-submission Date		Date Received 2nd submission	
Assessor Feedback: *Please note that constructive and useful feedback should allow students to understand: a) Strengths of performance b) Limitations of performance c) Any improvements needed in future assessments Feedback should be against the learning outcomes and assessment criteria to help students understand how these inform the process of judging the overall grade. Feedback should give full guidance to the students on how they have met the learning outcomes and assessment criteria.			
<i>I certify that to the best of my knowledge the evidence submitted for this assignment is the student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I have not solely used AI to grade the student's work.</i>			
Grade:	Assessor Signature:		Date:
Resubmission Feedback: *Please note resubmission feedback is focussed only on the resubmitted work			
Grade:	Assessor Signature:		Date:
Internal Verifier's Comments:			
Signature & Date:			

* Please note that grade decisions are provisional. They are only confirmed once internal and external moderation has taken place and grades decisions have been agreed at the assessment board.

STUDENT ASSESSMENT SUBMISSION AND DECLARATION

When submitting evidence for assessment, each student must sign a declaration confirming that the work is their own.

Student name: MN. Sihanass		Assessor name: Eng. A.L. Jubailah Begum	
Issue date: 03.03.2025	Submission date: 13.04.2025	Submitted on: 13.04.2025	
Programme: Data Structures & Algorithms			
Unit: 19			
Assignment number and title: Abstract Data Types in Object-Oriented Software: A Critical Examination			

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

Student declaration

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

Student signature: 1030371@bcas.ac

Date: 2025.04.13

General Guidelines

1. A Cover page or title page - You should always attach a title page to your assignment. Use previous page as your cover sheet and make sure all the details are accurately filled.
2. Attach this brief as the first section of your assignment.
3. All the assignments should be prepared using a word processing software.
4. All the assignments should be printed on A4 sized papers. Use single side printing.
5. Allow 1" for top, bottom, right margins and 1.25" for the left margin of each page.

Word Processing Rules

1. The font size should be 11 point, and should be in the style of Arial.
2. Use 1.5 line spacing. Left justify all paragraphs.
3. Ensure that all the headings are consistent in terms of the font size and font style.
4. Use footer function in the word processor to insert Your Name, Subject, Assignment No, and Page Number on each page. This is useful if individual sheets become detached for any reason.
5. Use word processing application spell check and grammar check function to help editing your assignment.

Important Points

1. It is strictly prohibited to use text boxes to add texts in the assignments, except for the compulsory information. eg: Figures, tables of comparison etc. Adding text boxes in the body except for the before mentioned compulsory information will result in rejection of your work.
2. Avoid using page borders in your assignment body.
3. Carefully check the hand in date and the instructions given in the assignment. Late submissions will not be accepted.
4. Ensure that you give yourself enough time to complete the assignment by the due date.
5. Excuses of any nature will not be accepted for failure to hand in the work on time.
6. You must take responsibility for managing your own time effectively.
7. If you are unable to hand in your assignment on time and have valid reasons such as illness, you may apply (in writing) for an extension.
8. Failure to achieve at least PASS criteria will result in a REFERRAL grade.
9. Non-submission of work without valid reasons will lead to an automatic REFERRAL. You will then be asked to complete an alternative assignment.
10. If you use other people's work or ideas in your assignment, reference them properly using HARVARD referencing system to avoid plagiarism. You have to provide both in-text citation and a reference list.
11. If you are proven to be guilty of plagiarism or any academic misconduct, your grade could be reduced to A REFERRAL or at worst you could be expelled from the course.

Unit 19: Data Structures & Algorithms

Assignment Brief

Programme Title	Pearson BTEC HND Computing
Student Name/ID Number	MN. Sihanass / 1030371
Unit Number and Title	Unit 19: Data Structures & Algorithms
Academic Year	2025
Unit Tutor	Eng. A.L. Jubailah Begum
Assignment Title	Abstract Data Types in Object-Oriented Software: A Critical Examination
Issue Date	10.02.2025
Submission Date	13.04.2025
Submission Format	
The submission should be in the form of an individual report written in a concise, formal business style using single spacing and font size 11. You are required to make use of headings, paragraphs and subsections as appropriate, and all work must be supported with research and referenced using Harvard referencing system. Please also provide an end list of references using the Harvard referencing system. The recommended word count is 5,000–5,500 words for the report excluding annexures, although you will not be penalised for exceeding the total word limit.	
Unit Learning Outcomes	
LO1 Examine abstract data types, concrete data structures and algorithms LO2 Specify abstract data types and algorithms in a formal notation LO3 Implement complex data structures and algorithms LO4 Assess the effectiveness of data structures and algorithms.	
Transferable skills and competencies developed	

Vocational scenario

Scenario: Abstract Data Types in Object-Oriented Software: A Critical Examination

You've recently joined an IT company as a junior software developer specializing in data structures and algorithms. As part of your role, you collaborate closely with the team responsible for object-oriented software development.

However, you've encountered a disagreement with one of the senior developers who holds the belief that Abstract Data Types (ADTs) do not play a significant role in object-oriented software development. In contrast, you firmly believe that ADTs serve as a fundamental concept in this context, providing a structured approach to modeling data.

Seeking clarification and resolution, you approach your team lead for guidance. After discussing the matter with your team lead, they advise you to conduct thorough research to gain a deeper understanding of the role of ADTs in object-oriented software development.

Your task is to conduct comprehensive research and compile a detailed report that investigates the significance of ADTs in object-oriented software development. Through your research findings, you aim to reconcile the differing perspectives within the team and provide insights that will contribute to a better understanding of the relationship between ADTs and object-oriented software development practices within the company.

Assignment activity and guidance

Part 1	Examine abstract data types, concrete data structures and algorithms
1	The team lead has asked you to create a report to help your team explaining the following 1. What are the common abstract data types that can be created/implemented? 2. How to create them? 3. What are the operations they can carry out in the particular data structure like inserting, deleting, updating, searching, sorting, etc...?
2	1. The team lead has asked you to help them to create a report to explain the operations how the Random-access memory saves function calls in a computer for execution using a memory stack.

3	- The team lead has asked you to create and do a demonstration to explain the queue and how the first in first out (FIFO) is executed in a queue by creating a physical tangible object or objects or a visible graphical illustration: - Example using plastic cups as an array and Ping-Pong balls as the data which are inserted then removed. Or any other tangible physical examples. - The demonstration should explain the FIFO in a queue very clearly and effectively to your software team. The illustration should be recorded as a video and submitted along with the report - The team lead has sent you the java program containing multiple sorting algorithms that can sort an array, using the program given explain to the developers the best sorting algorithm by comparing the time each algorithm takes to sort 100, 1000, 10000,100000 random numbers.
4	- The team lead has provided the following network algorithms to you - Bellman Ford's Algorithm - Dijkstra's Algorithm - Floyd-Warshall's Algorithm - Johnson's Algorithm Select two of these algorithms and create visual illustration. You can use any designing software or you can draw them in paper, create both of the algorithm which you chose from the above list using a sample scenario of your own, then explain the operations of these two algorithms by explaining the sample scenario and step by step how the algorithm will find the shortest path between two nodes. Submit your illustrations and the explanation as a power point presentation.
Part 2	Specify abstract data types and algorithms in a formal notation
1	Specify the abstract data type for a software stack. Then examine and explain the advantages of information hiding and the encapsulation used in abstract data types, you can use the software stack as an example to explain the advantages.

2

Explain what the core concepts of object-oriented programming, (Encapsulation, Abstraction, Inheritance and polymorphism) then how these concepts are connected to the abstract data types, if they are similar in creation and usage that proves that the ADT's are the basics of OOP, state whether you agree and say why?

Scenario: Abstract Data Types in Object-Oriented Software: Music Playlist Management System

You've recently joined an IT company as a junior software developer specializing in data

Since you have demonstrated a solid understanding of ADT and OOP, your team lead is impressed, and they have promoted you to the position of associate software engineer. You have been assigned to a project to create a prototype music playlist management system, leveraging Abstract Data Types (ADTs) to organize and manage music files effectively.

As an associate software developer, your role is to design and develop the prototype music playlist management system. The system will utilize an ADT, such as a Stack, Linked List, or Queue, to represent and manage the music files within the playlist.

The objective of the system is to allow users to create, edit, and manage playlists consisting of audio files. Users should be able to add new songs to the playlist, remove existing ones, rearrange the order of songs, and play the playlist sequentially or in a shuffled order.

Key Features of the Music Playlist Management System:

- **Playlist Creation:** Users can create new playlists and give them descriptive names.
- **Song Management:** Users can add songs to the playlist, remove songs, and rearrange the order of songs.
- **Playback Options:** Users can choose to play the playlist sequentially or in a shuffled order.
- **User Interface:** The system should have an intuitive user interface that allows users to interact with the playlist easily.
- **ADT Implementation:** Utilize an appropriate ADT, such as a Stack, Linked List, or Queue, to represent the music files within the playlist.

Through the use of ADTs, you will organize and manage the music files within the playlist, laying the foundation for future enhancements and optimizations.

Part 3	Implement complex data structures and algorithms
1	Implement appropriate ADT's and algorithms to achieve the required output for the above scenario, create a test case report and test your program and submit the program coding and test report.
2	Prepare a presentation containing the demonstration of the code and algorithm you created and create a comprehensive survey to test the complexity of the algorithm you have created. Demonstrate your program to other developers in your class using the report, explain what ADT's are used, how they are used with the algorithms, how is the algorithm designed and how it achieves the desired output? Then distribute and the survey you have created and conduct the survey among your class to understand and determine how complex is your solution and how accurate it is?
Part 4	Assess the effectiveness of data structures and algorithms
1	Explain what asymptotic analysis is and how asymptotic analysis can be used to assess the effectiveness of an algorithm. Determine two ways in which the efficiency of an algorithm can be measured
2	Create three different programs 1. Containing array as the main ADT 2. Containing Linked list as the main ADT 3. Containing Stack as the ADT Run the programs and find out the compromise of each ADT's consider development time, implementation time, runtime, and accuracy. Critically, a good abstract data type should be representation independent. This means that the use of an abstract type is independent of its representation (the actual data structure or data fields used to implement it), so that changes in representation have no effect on code outside the abstract type itself. For example, the operations offered by List are independent of whether the list is represented as a linked list or as an array. Evaluate three benefits of using implementation independent data structures.
Recommended Resources Please note that the resources listed are examples for you to use as a starting point in your research – the list is not definitive.	

Learning Outcomes and Assessment Criteria

Pass	Merit	Distinction
LO1 Examine abstract data types, concrete data structures and algorithms		
P1 Create a design specification for data structures, explaining the valid operations that can be carried out on the structures. P2 Determine the operations of a memory stack and how it is used to implement function calls in a computer.	M1 Illustrate, with an example, a concrete data structure for a First in First out (FIFO) queue. M2 Compare the performance of two sorting algorithms.	
LO2 Specify abstract data types and algorithms in a formal notation		
P3 Specify the abstract data type for a software stack using an imperative definition.	M3 Examine the advantages of encapsulation and information hiding when using an ADT.	D2 Discuss the view that imperative ADTs are a basis for object orientation offering a justification for the view.
LO3 Implement complex data structures and algorithms		

P4 Implement a complex ADT and algorithm in an executable programming language to solve a well-defined problem. P5 Implement error handling and report test results	M4 Demonstrate how the implementation of an ADT/algorithm solves a well-defined problem.	D3 Critically evaluate the complexity of an implemented ADT/algorithm.
L04 Assess the effectiveness of data structures and algorithms		
P6 Discuss how asymptotic analysis can be used to assess the effectiveness of an algorithm. P7 Determine two ways in which the efficiency of an algorithm can be measured, illustrating your answer with an example.	M5 Interpret what a trade-off is when specifying an ADT, using an example to support your answer.	D4 Evaluate three benefits of using implementation independent data structures.

Acknowledgement

I would like to express my sincere gratitude to my instructor, Eng. A.L. Jubailah Begum, for their invaluable guidance and support throughout the completion of this assignment. Their constructive feedback and insightful recommendations have greatly contributed to my understanding of Data Structures and Algorithms. Additionally, I extend my appreciation to BCAS Kalmunai Campus and Pearson BTEC HND in Computing for providing a well-structured curriculum that enhances practical knowledge. Lastly, I am grateful to my family and friends for their continuous encouragement and motivation during this learning journey.

Abstract

This report provides an in-depth exploration of Abstract Data Types (ADTs) and their significance within object-oriented software development. It addresses the common types of ADTs, their construction, the operations they support, and their practical applications in real-world programming scenarios. The report also covers an analysis of algorithm effectiveness using asymptotic analysis, and the evaluation of efficiency metrics such as time and space complexity.

To demonstrate practical understanding, the report includes the development of three Python programs using Arrays, Linked Lists, and Stacks as primary ADTs. A comparison is made based on development time, implementation complexity, runtime, and accuracy. This research aims to clarify the importance of ADTs in modern software systems and highlight the trade-offs inherent in choosing specific data structures for problem-solving.

Table of Contents

Introduction	1
1. Examine abstract data types, concrete data structures and algorithms	2
1.1.1 Common Abstract Data Types (ADTs):.....	2
1.1.2 How to Create Them	4
1.1.3 Operations in the ADTs	5
1.2.1 How RAM Manages Function Calls	6
1.3.2 Comparing sorting algorithms on different scenarios.	9
Time Complexity Overview.....	9
Performance Analysis by Input Size	9
Final Recommendations	10
2. Specify abstract data types and algorithms in a formal notation	11
2.1 Abstract Data Type for a Software Stack and the Advantages of Information Hiding and Encapsulation	11
2.1.1 Advantages of Information Hiding and Encapsulation in ADTs:	11
2.2 Exploring the Core Concepts of Object-Oriented Programming and Their Relationship with Abstract Data Types	12
2.2.1 Encapsulation	12
2.2.2 Abstraction	12
2.2.3 Inheritance	13
2.2.4 Polymorphism	13
3. Implement complex data structures and algorithms.....	14
3.1 Music Playlist System using appropriate ADT	14
Test Case Report	17
Test case summary	21
4. Assess the effectiveness of data structures and algorithms.....	22
4.1 Understanding Asymptotic Analysis and Algorithm Efficiency	22
4.1.1 Asymptotic Analysis: Definition and Purpose	22
4.1.2 Assessing Algorithm Effectiveness Through Asymptotic Analysis	24
4.1.3 Measuring Algorithm Efficiency: Two Key Metrics	25
4.2 Comparative Analysis of Array, Linked List, and Stack ADTs	27
4.2.1 Program Implementations	27
4.2.2 Trade-Off Analysis	30
4.3.3 Understanding Trade-Offs in ADT Selection	31
Conclusion	32
References.....	33

List of Tables

Table 1: Time complexity overview.....	9
Table 2: Recommendations to choose best sorting algorithm to every scenario.....	10
Table 3: test case summary	21

List of Figures

Figure 1: Stack data structure (Geeksforgeeks, 2025).....	2
Figure 2: Queue data structure (geeksforgeeks, 2015).....	2
Figure 3: tree data structure (Geeksforgeeks, 2025)	3
Figure 4: Binary tree data structure (geeksforgeeks, 2025)	3
Figure 5: Binary search tree (geeksforgeeks, 2025)	4
Figure 6: graph data structure (S, 2025).....	4
Figure 7: How RAM Manages Function Calls	8
Figure 8: code segment to implement the queue ADT to create a music playlist system part 1.....	15
Figure 9: code segment to implement the queue ADT to create a music playlist system part 2.....	16
Figure 10: add a song to playlist.....	17
Figure 11: song added to the playlist successfully.....	17
Figure 12: remove a song from the playlist.....	18
Figure 13: song remove from the playlist successfully.....	18
Figure 14: displaying the current playlist successfully.....	19
Figure 15: playing the playlist sequentially	19
Figure 16: play the playlist in shuffled mode	20
Figure 17: removing a song from empty playlist and getting warning.....	20
Figure 18: Big O (geeksforgeeks, 2024)	22
Figure 19: Omega (geeksforgeeks, 2024)	23
Figure 20: Theta (geeksforgeeks, 2024).....	23
Figure 21: Time Complexity illustration (schools, 2025).....	25
Figure 22: space complexity illustration (krahets, 2025)	26
Figure 23: creating a program containing array as main ADT	27
Figure 24: creating a program containing linked list as a main ADT	28
Figure 25: creating a program containing stack as a main ADT	29

Introduction

Abstract Data Types (ADTs) form the conceptual backbone of data organization in computer science. Unlike concrete data structures, ADTs focus on the "what" rather than the "how" defining a set of operations and behaviors without specifying implementation details. This separation of concerns enables flexibility, reusability, and modularity in software design.

In object-oriented programming (OOP), the principles of abstraction, encapsulation, inheritance, and polymorphism align closely with the structure and purpose of ADTs. When software developers choose an ADT for a specific problem, they balance trade-offs related to time complexity, space usage, ease of implementation, and scalability. This report critically examines the significance of ADTs through theory and hands-on programming exercises involving Arrays, Linked Lists, and Stacks. It also highlights how asymptotic analysis can measure algorithm performance and discusses the benefits of implementation-independent data structures in software engineering.

1. Examine abstract data types, concrete data structures and algorithms

1.1.1 Common Abstract Data Types (ADTs):

Abstract Data Types (ADTs) are theoretical models for data structures that define the operations that can be performed on a collection of data, but they do not specify the underlying implementation. The most used ADTs include:

Stack: A Stack is a sequential data structure that manages data based on either a Last-In, First-Out (LIFO) or First-In, Last-Out (FILO) principle. In LIFO, the most recently added item is removed first, while FILO means the earliest added item is removed first (Geeksforgeeks, 2025).

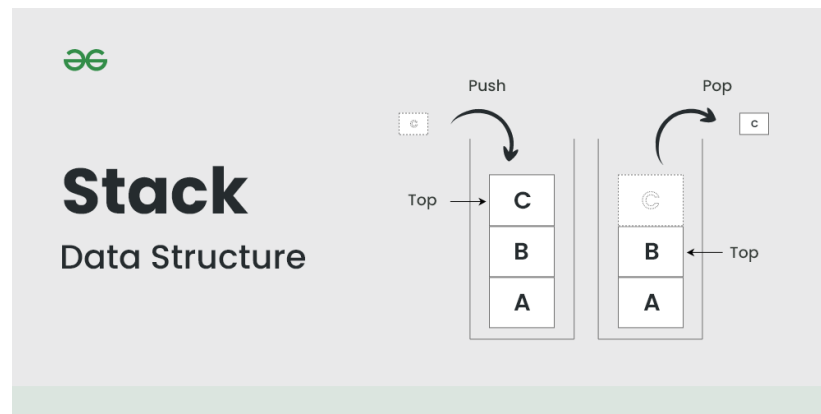


Figure 1: Stack data structure (Geeksforgeeks, 2025)

Queue: A Queue is a core data structure where data is organized such that the first element inserted is also the first element extracted, following a FIFO order. Its simplicity and efficiency make it widely applicable (geeksforgeeks, 2015).

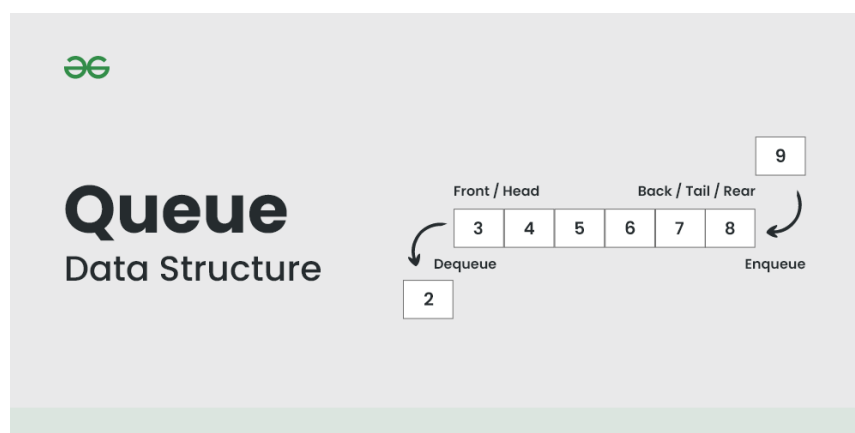


Figure 2: Queue data structure (geeksforgeeks, 2015)

Tree: A tree data structure is a non-linear method of storing information, where nodes are interconnected by edges, and a single, distinct path exists between all nodes (Geeksforgeeks, 2025).

Tree

Data Structure

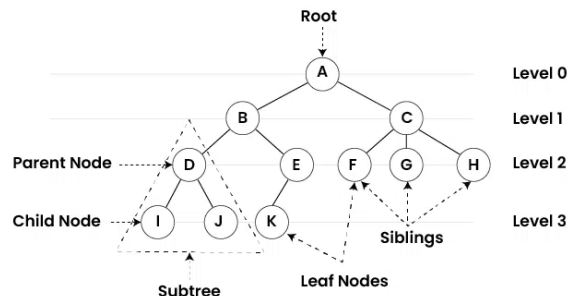


Figure 3: tree data structure (Geeksforgeeks, 2025)

Binary Tree: A Binary Tree is a tree-like data structure where each node can have a maximum of two branches, known as the left and right children. It's widely used for organizing and accessing data effectively, supporting operations like adding, removing, and navigating through elements (geeksforgeeks, 2025).

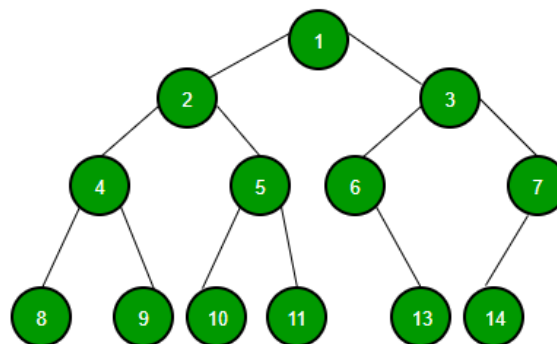


Figure 4: Binary tree data structure (geeksforgeeks, 2025)

Binary Search Tree (BST): A Binary Search Tree is a hierarchical data structure designed to store data in a sorted order. Each node has up to two children, with the left child holding values smaller than the parent and the right child holding larger values. This organization enables quick searching, insertion, and deletion of data (geeksforgeeks, 2025).

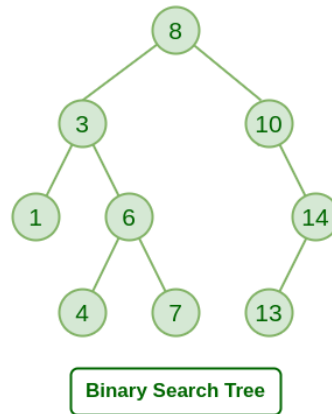


Figure 5: Binary search tree (geeksforgeeks, 2025)

Graph: A collection of nodes (vertices) connected by edges, which can be directed or undirected (Thomas H Cormen. Charles Eric Leiserson, 2009).

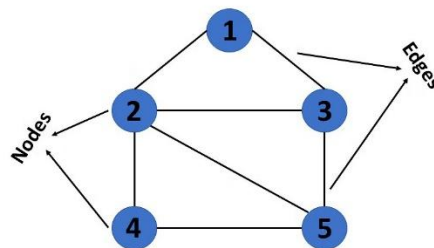


Figure 6: graph data structure (S, 2025)

1.1.2 How to Create Them

The implementation of ADTs varies depending on the data structure. Below is an outline of how these ADTs are typically created:

1. Stack:

- **Implementation:** A stack can be implemented using an array or a linked list. It provides two main operations: **push** (insert an element) and **pop** (remove an element).
- **Creation:** Define a class or structure for the stack with methods for **push**, **pop**, **peek**, and **isEmpty**.

2. Queue:

- **Implementation:** A queue can be implemented using an array or a linked list. It supports two main operations: **enqueue** (add an element) and **dequeue** (remove an element).
 - **Creation:** Define a class or structure with methods for **enqueue**, **dequeue**, **front**, **isEmpty**.
3. **Tree:**
- **Implementation:** A tree can be implemented using nodes where each node points to its children.
 - **Creation:** Define a node class with references to child nodes. A tree class is created to manage the structure (insertion, traversal, etc.).
4. **Binary Tree:**
- **Implementation:** Similar to a tree but with specific rules regarding the number of children (at most two).
 - **Creation:** Define a node class with pointers to left and right child nodes. The tree class will handle insertion, traversal, and deletion.
5. **Binary Search Tree (BST):**
- **Implementation:** A binary tree with an added constraint: left child < parent node < right child.
 - **Creation:** Define a node class with left and right pointers. The tree class should include functions for insertion, searching, deletion, and traversal (in-order, pre-order, post-order).
6. **Graph:**
- **Implementation:** A graph can be implemented using adjacency matrices or adjacency lists, depending on whether it's a dense or sparse graph.
 - **Creation:** Define a graph class with methods for adding vertices and edges. For an adjacency list, use lists or hash maps to represent edges (GeeksforGeeks, 2014).

1.1.3 Operations in the ADTs

Each ADT has a specific set of operations that can be performed on it. Here are some common operations:

1. **Stack:**
- **Push:** Add an element to the top of the stack.
 - **Pop:** Remove the top element from the stack.
 - **Peek:** View the top element without removing it.

- **isEmpty:** Check if the stack is empty (programiz).
2. **Queue:**
 - **Enqueue:** Add an element to the rear of the queue.
 - **Dequeue:** Remove an element from the front of the queue.
 - **Front:** View the front element without removing it.
 - **isEmpty:** Check if the queue is empty (programiz).
 3. **Tree:**
 - **Insert:** Add a node to the tree.
 - **Delete:** Remove a node from the tree.
 - **Search:** Find a node in the tree.
 - **Traversal:** Visit all nodes (in-order, pre-order, post-order).
 - **Update:** Modify data in a node (GeeksforGeeks, 2025).
 4. **Binary Tree:**
 - **Insert:** Add a node to the tree.
 - **Delete:** Remove a node from the tree.
 - **Search:** Search for an element.
 - **Traversal:** In-order, pre-order, post-order traversal.
 5. **Binary Search Tree (BST):**
 - **Insert:** Insert a node in the correct position to maintain the BST property.
 - **Delete:** Remove a node while maintaining the BST property.
 - **Search:** Find an element (by comparing with node values).
 - **Traversal:** In-order traversal to access elements in sorted order.
 - **FindMin/FindMax:** Find the smallest/largest element (GeeksforGeeks, 2025).
 6. **Graph:**
 - **AddVertex:** Add a new vertex to the graph.
 - **AddEdge:** Add an edge between two vertices.
 - **RemoveVertex:** Remove a vertex and its associated edges.
 - **RemoveEdge:** Remove an edge between two vertices.
 - **Search:** Perform a search (DFS/BFS).
 - **Traversal:** Traverse the graph using DFS or BFS.
 - **Update:** Modify vertex or edge data.

1.2.1 How RAM Manages Function Calls

One critical use of RAM is managing function calls through a structure known as the **call stack**. This mechanism ensures that each function operates correctly and returns control to the appropriate location in the program.

Function Calls and the Call Stack

When a function is invoked, the system needs to keep track of several pieces of information to ensure proper execution and return. This is managed using the call stack, a **Last-In-First-Out (LIFO)** data structure that operates as follows:

1. **Stack Frame Creation:** Each time a function is called, a new *stack frame* is created and pushed onto the top of the stack. This frame typically contains:
 - **Return Address:** The location in the code to return to after the function completes.
 - **Function Parameters:** The arguments passed to the function.
 - **Local Variables:** Variables declared within the function.
2. **Function Execution:** With its stack frame in place, the function executes, utilizing the parameters and local variables stored in its frame.
3. **Function Return:** Upon completion, the function's stack frame is popped from the stack, and control returns to the address specified in the return address, resuming execution of the calling function.

The call stack follows a Last-In-First-Out (LIFO) principle. That means the most recently called function is the first one to complete and be removed (or "popped") from the stack. When a function completes its execution, its stack frame is removed, and the system uses the stored return address to jump back to the calling function.

Below is an illustrative diagram that shows how function calls are stored on the call stack:

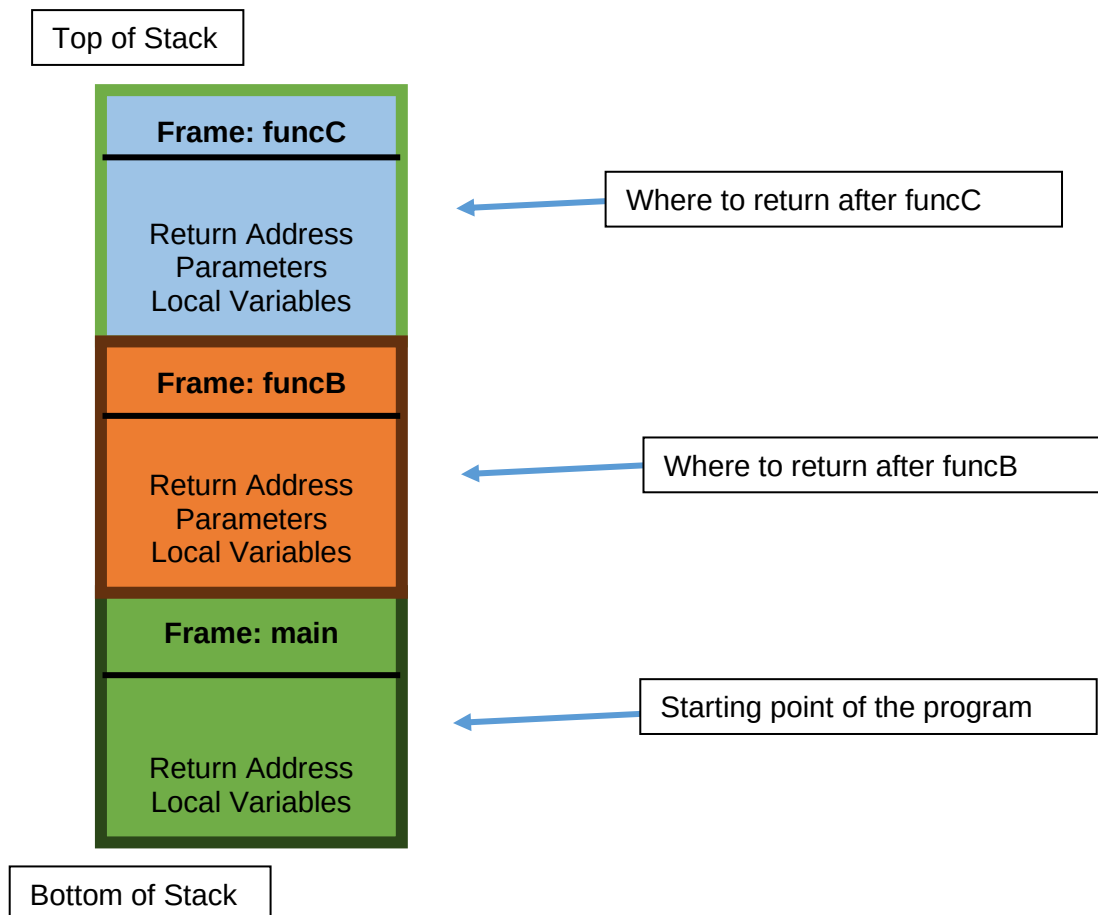


Figure 7: How RAM
Manages Function Calls

How It Works in Practice

1. **Function Call (Push):** When `main()` calls `funcB()`, a new frame for `funcB` is pushed onto the stack. If `funcB()` then calls `funcC()`, a new frame for `funcC` is added on top.
2. **Function Execution:** The CPU executes the code within the active function (i.e., the one at the top of the stack).
3. **Function Return (Pop):** When `funcC()` finishes executing, its frame is popped from the stack, and the CPU retrieves the return address to continue execution in `funcB()`. This pop operation occurs each time a function call is completed.
4. **Restoration of Context:** The popped frame may also contain saved register values, which are restored so that the calling function resumes with the same context it had before making the call.

This stack-based approach is fundamental in managing nested function calls, recursion, and ensuring that every function returns control to the correct location in the code.

1.3.1 FIFO illustration for queue ADT is attached along with assignment.

1.3.2 Comparing sorting algorithms on different scenarios.

To determine the most efficient sorting algorithm for arrays of sizes 100, 1,000, 10,000, and 100,000, we can analyze the theoretical time complexities of various sorting algorithms.

Time Complexity Overview

Sorting Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stability
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Table 1: Time complexity overview

Performance Analysis by Input Size

1. Small Arrays ($n = 100$)

- **Insertion Sort:** Efficient due to its $O(n)$ best-case performance on nearly sorted data and low overhead.
- **Bubble Sort:** Simple but generally slower than Insertion Sort.
- **Selection Sort:** Consistently $O(n^2)$, making it less efficient even for small datasets.
- **Merge, Quick, Heap Sorts:** Offer $O(n \log n)$ performance but may have higher overhead, making them less optimal for very small arrays (GeeksforGeeks, 2019).

Recommendation: Use **Insertion Sort** for small arrays due to its simplicity and efficiency.

2. Medium Arrays (n = 1,000)

- **Insertion, Bubble, Selection Sorts:** All have $O(n^2)$ time complexity, leading to significant performance degradation.
- **Merge, Quick, Heap Sorts:** With $O(n \log n)$ complexity, these algorithms handle medium-sized arrays more efficiently (GeeksforGeeks, 2019).

Recommendation: Opt for **Merge Sort**, **Quick Sort**, or **Heap Sort** for better performance on medium arrays.

3. Large Arrays (n = 10,000 and 100,000)

- **Insertion, Bubble, Selection Sorts:** $O(n^2)$ complexity makes these algorithms impractical for large datasets.
- **Merge Sort:** Consistent $O(n \log n)$ performance and stability make it suitable for large arrays, especially when stability is required.
- **Quick Sort:** Generally faster in practice due to low overhead, but worst-case $O(n^2)$ complexity can be a concern without proper pivot selection.
- **Heap Sort:** Offers $O(n \log n)$ performance with $O(1)$ space complexity, making it efficient for large datasets where memory usage is a concern (GeeksforGeeks, 2019).

Recommendation: For large arrays, **Merge Sort** is preferred when stability is important, **Quick Sort** for general-purpose sorting with good average performance, and **Heap Sort** when memory usage needs to be minimized.

Final Recommendations

Input Size	Recommended Algorithm(s)	Justification
100	Insertion Sort	Efficient for small datasets with low overhead.
1,000	Merge Sort / Quick Sort / Heap Sort	Better performance due to $O(n \log n)$ complexity.
10,000	Merge Sort / Quick Sort / Heap Sort	Efficient handling of larger datasets with optimal time complexity.
100,000	Merge Sort / Quick Sort / Heap Sort	Suitable for very large datasets; choice depends on specific requirements.

Table 2: Recommendations to choose best sorting algorithm to every scenario

for small datasets, **Insertion Sort** is optimal due to its simplicity and efficiency. For larger datasets, algorithms with $O(n \log n)$ complexity, such as **Merge Sort**, **Quick Sort**, and **Heap Sort**, are more suitable. The choice among these depends on factors like the need for stability and memory constraints.

1.4.1 shortest path algorithm slide is attached along with assignment.

2. Specify abstract data types and algorithms in a formal notation

2.1 Abstract Data Type for a Software Stack and the Advantages of Information Hiding and Encapsulation

A stack is a fundamental Abstract Data Type (ADT) that operates on the Last-In-First-Out (LIFO) principle (user1849655, 2012). This means the most recently added element is the first to be removed. The stack ADT supports the following primary operations:

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes and returns the top element of the stack.
- **Peek/Top:** Returns the top element without removing it.
- **IsEmpty:** Checks if the stack is empty.
- **Size:** Returns the number of elements in the stack (pythonds).

The internal implementation details, such as whether the stack uses an array or a linked list, are hidden from the user. This abstraction allows users to interact with the stack through its defined operations without needing to understand its underlying structure (Klukowska).

2.1.1 Advantages of Information Hiding and Encapsulation in ADTs:

In the context of Abstract Data Types like the stack, **information hiding**, and **encapsulation** offer several benefits:

1. **Modularity:** Encapsulation groups data and the methods that operate on that data into a single unit. This modularity makes the system more organized and manageable. For instance, the stack's implementation details are encapsulated within its definition, allowing developers to work with a clear and concise interface (Alex, 2023).
2. **Maintainability:** By hiding the internal workings of the stack, changes can be made to its implementation without affecting the code that uses the stack. This

separation of concerns simplifies maintenance and reduces the risk of introducing errors when modifications are necessary.

3. **Reusability:** A well-defined stack ADT can be reused across different programs and projects. Since the implementation details are hidden, the same stack interface can be implemented in various ways (e.g., using arrays or linked lists) without altering the way other parts of the program interact with it.
4. **Security:** Information hiding restricts access to the internal data of the stack, preventing unintended interference and ensuring that the stack operates as intended. This protection is crucial in maintaining data integrity and consistency within the application (EnableGeek, 2024).

By leveraging information hiding and encapsulation, Abstract Data Types like the stack provide a robust and flexible foundation for building reliable software systems.

2.2 Exploring the Core Concepts of Object-Oriented Programming and Their Relationship with Abstract Data Types

Object-Oriented Programming (OOP) is a paradigm centered around objects that encapsulate data and behaviour. Its core principles Encapsulation, Abstraction, Inheritance, and Polymorphism form the foundation for designing modular and maintainable software.

2.2.1 Encapsulation

Encapsulation involves bundling data (attributes) and methods (functions) that operate on that data into a single unit, typically a class. This approach restricts direct access to some of an object's components, safeguarding data integrity and promoting modularity. For instance, in a BankAccount class, the account balance can be kept private, accessible only through deposit and withdrawal methods, ensuring controlled manipulation (Team, 2024).

2.2.2 Abstraction

Abstraction focuses on hiding complex implementation details and showcasing only the essential features of an object. It allows developers to interact with high-level interfaces without needing to understand the intricate workings beneath. For example, when using a List in programming, one can add or remove elements without delving into the underlying data structure operations (Team, 2024).

2.2.3 Inheritance

Inheritance is a mechanism where a new class, known as a subclass or derived class, inherits attributes and methods from an existing class, referred to as a superclass or base class. This promotes code reusability and establishes a hierarchical relationship between classes. For example, a Vehicle class can have subclasses like Car and Motorcycle, each inheriting common properties like speed and capacity while introducing specific features (Team, 2024).

2.2.4 Polymorphism

Polymorphism allows objects of different classes to respond to the same method call in varied ways. It enables a single interface to represent different underlying data types, enhancing flexibility and integration. For instance, a function drawShape() can invoke drawing methods on various shape objects like circles or rectangles, each rendering appropriately based on its specific implementation (Team, 2024).

Yes, I agree that **Abstract Data Types (ADTs) are the foundation of Object-Oriented Programming (OOP)** to some extent. ADTs establish the concept of **encapsulation** and **abstraction**, which are fundamental to OOP. However, ADTs alone do not fully encompass **inheritance** and **polymorphism**, which are key distinguishing features of OOP (Editorial, 2025).

Why I Agree:

1. Encapsulation & Abstraction in ADTs - Core to OOP:

- ADTs define **what operations** can be performed without revealing **how** they are implemented, just like OOP classes hide internal details and expose only necessary functionalities.
- Example: A **Stack ADT** specifies push() and pop() but doesn't dictate whether it uses an **array or a linked list**, just like an OOP class hides its internal structure.

2. Data-Centric Approach:

- ADTs focus on **defining structured data** and associated operations, which is similar to how OOP classes encapsulate state and behavior.

3. Foundation for OOP Evolution:

- ADTs came before OOP and influenced its development. Early OOP concepts built upon ADT principles, adding **inheritance** and **polymorphism** for better reusability and flexibility (Editorial, 2025).

3. Implement complex data structures and algorithms

3.1 Music Playlist System using appropriate ADT

For this scenario, I choose queue ADT. Because A Queue is ideal for managing a playlist where songs are played in the order they are added (FIFO). This structure ensures the first song added is the first to be played similar to how many music queues work in real-life streaming services.

```

import tkinter as tk
from tkinter import messagebox, simpledialog
import random

# Queue ADT Implementation
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def peek(self):
        return self.items[0] if not self.is_empty() else None

    def get_all_songs(self):
        return self.items.copy()

    def shuffle(self):
        random.shuffle(self.items)

# GUI using Tkinter
class PlaylistApp:
    def __init__(self, master):
        self.master = master
        master.title("Music Playlist Management System")
        master.geometry("500x400")
        master.config(bg="#ececce")

        self.playlist = Queue()

        self.label = tk.Label(master, text="Music Playlist", font=("Arial", 16), bg="#ececce")
        self.label.pack(pady=10)

        self.song_listbox = tk.Listbox(master, width=50, height=10)
        self.song_listbox.pack(pady=10)

        self.add_button = tk.Button(master, text="Add Song", command=self.add_song)
        self.add_button.pack(pady=5)

        self.remove_button = tk.Button(master, text="Remove Song", command=self.remove_song)
        self.remove_button.pack(pady=5)

        self.play_button = tk.Button(master, text="Play Playlist (Sequential)", command=self.play_playlist)
        self.play_button.pack(pady=5)

        self.shuffle_button = tk.Button(master, text="Shuffle Playlist", command=self.shuffle_playlist)
        self.shuffle_button.pack(pady=5)

```

Figure 8: code segment to implement the queue ADT to create a music playlist system part 1

```

self.shuffle_button = tk.Button(master, text="Shuffle Playlist", command=self.shuffle_playlist)
self.shuffle_button.pack(pady=5)

self.refresh_button = tk.Button(master, text="Refresh Playlist View", command=self.refresh_listbox)
self.refresh_button.pack(pady=5)

def add_song(self):
    song = simpledialog.askstring("Add Song", "Enter song name:")
    if song:
        self.playlist.enqueue(song)
        self.refresh_listbox()

def remove_song(self):
    if not self.playlist.is_empty():
        removed = self.playlist.dequeue()
        messagebox.showinfo("Removed Song", f"Removed: {removed}")
        self.refresh_listbox()
    else:
        messagebox.showwarning("Warning", "The playlist is empty!")

def play_playlist(self):
    if self.playlist.is_empty():
        messagebox.showwarning("Warning", "No songs to play!")
    else:
        songs = self.playlist.get_all_songs()
        messagebox.showinfo("Playing Playlist", "\n".join(songs))

def shuffle_playlist(self):
    self.playlist.shuffle()
    self.refresh_listbox()

def refresh_listbox(self):
    self.song_listbox.delete(0, tk.END)
    for song in self.playlist.get_all_songs():
        self.song_listbox.insert(tk.END, song)

if __name__ == "__main__":
    root = tk.Tk()
    app = PlaylistApp(root)
    root.mainloop()

```

Figure 9: code segment to implement the queue ADT to create a music playlist system part 2

Note: The implementation code attached along with the assignment separately.

Test Case Report & evidences

1. Test Case 1: add songs to the playlist

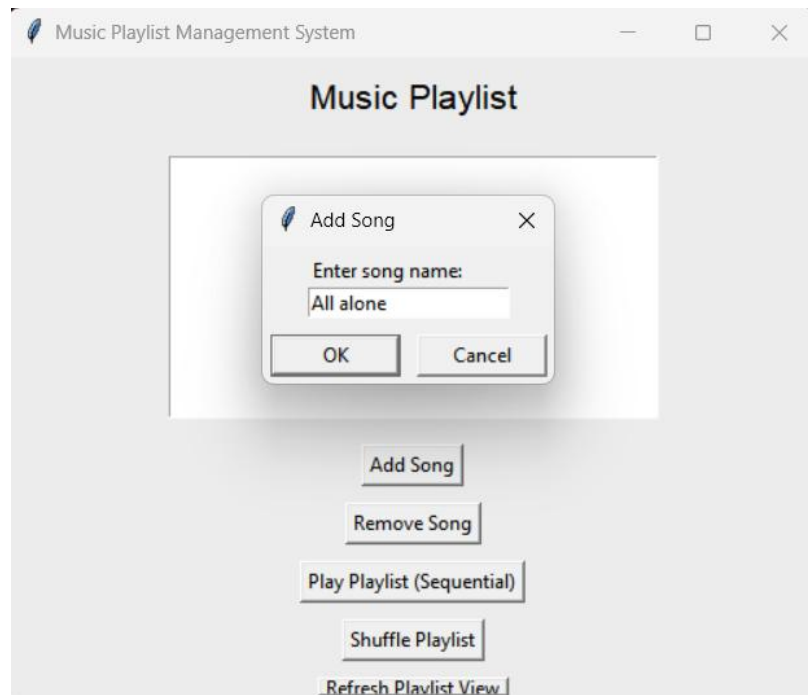


Figure 10: add a song to playlist

Output: songs added successfully to the playlist

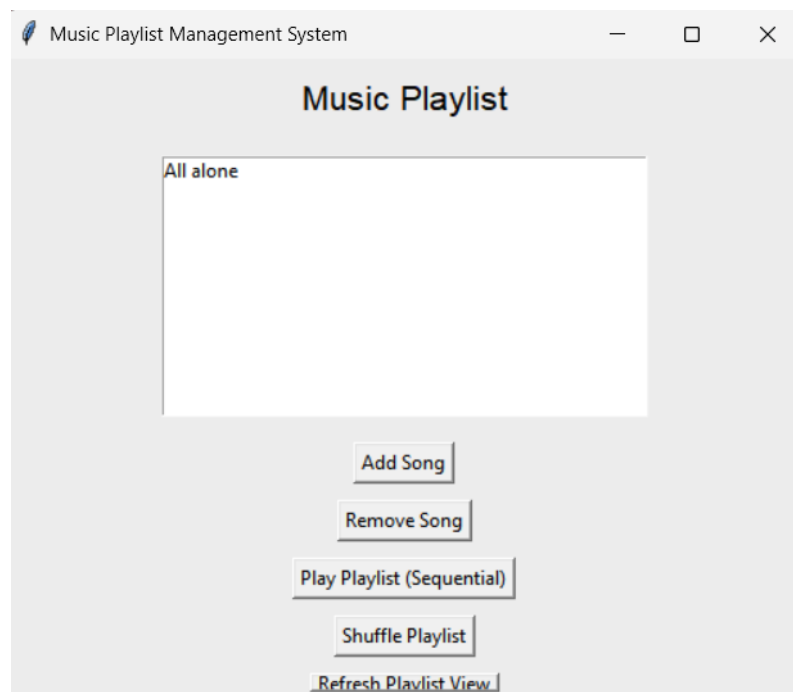


Figure 11: song added to the playlist successfully

2. Test Case 2: remove a song from the playlist

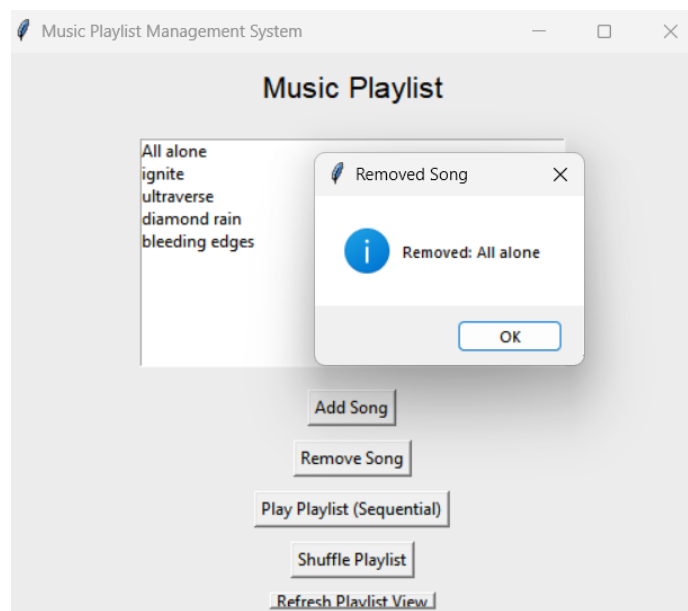


Figure 12: remove a song from the playlist

Output: first song removed successfully from the playlist

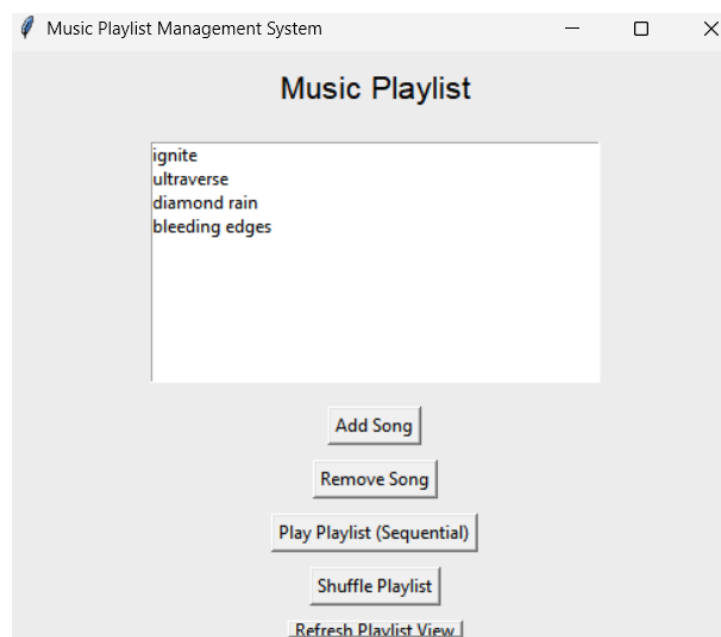


Figure 13: song remove from the playlist successfully

3. Test Case 3: Add multiple songs and view playlist

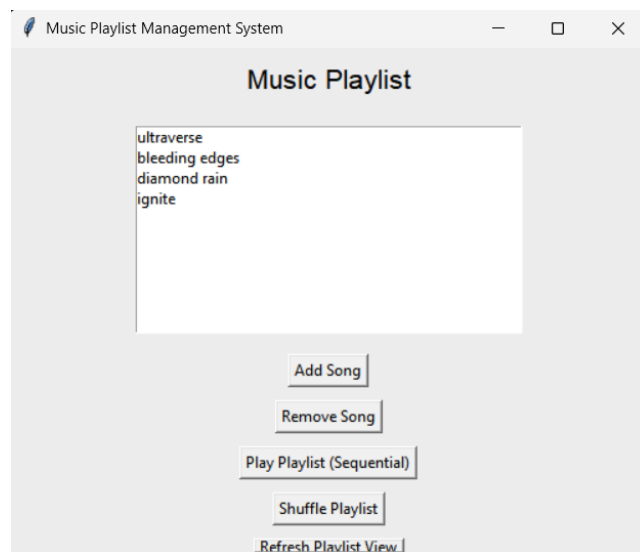


Figure 14: displaying the current playlist successfully

4. Test Case 4: playing playlist sequentially

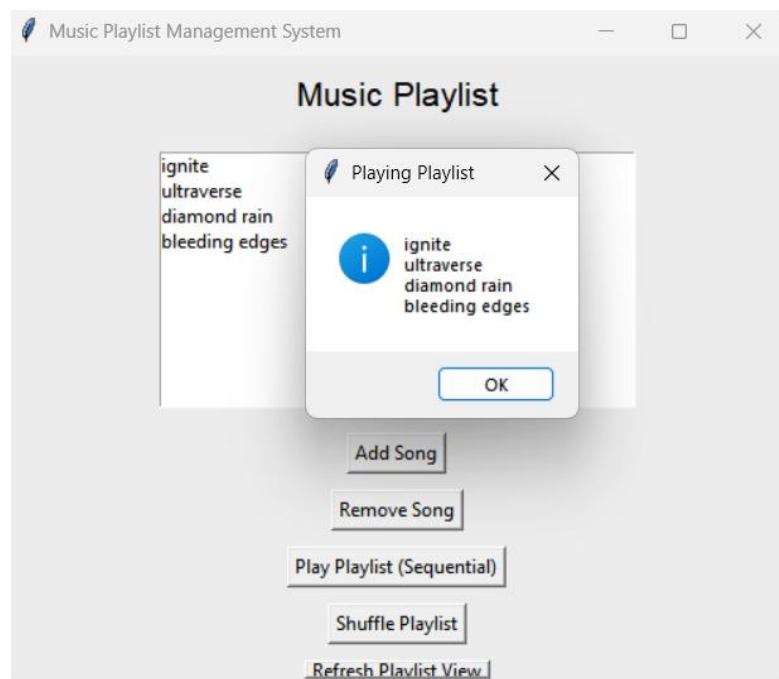


Figure 15: playing the playlist sequentially

5. Test Case 5: playing playlist shuffled

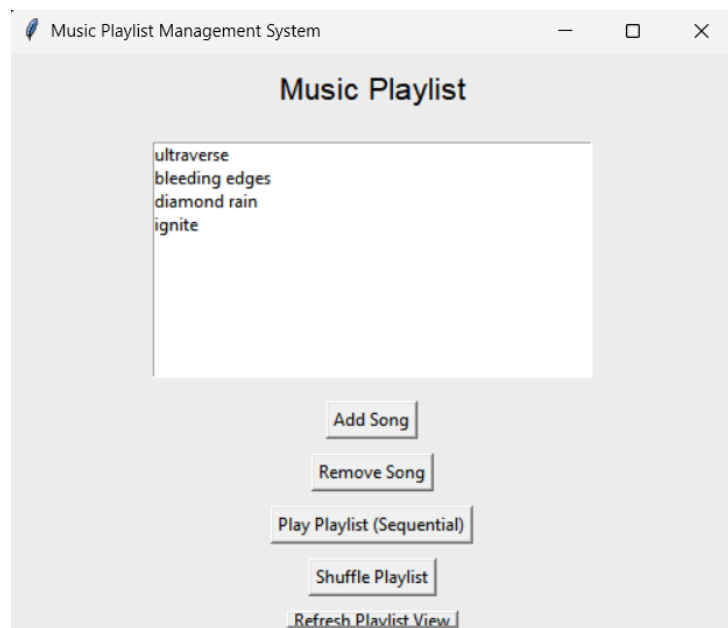


Figure 16: play the playlist in shuffled mode

6. Test Case 6: remove a song from an empty playlist

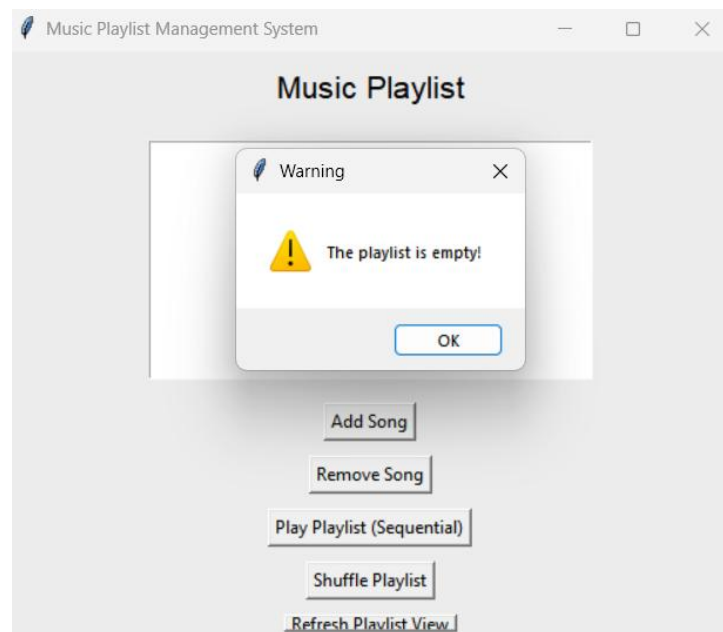


Figure 17: removing a song from empty playlist and getting warning

Test case summary

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC01	Add song to playlist	"All alone"	Song added to the list	Song added correctly	Pass
TC02	Remove song from playlist	First song in queue	First song removed	First song removed	Pass
TC03	Add multiple songs and view playlist	"ultraverse", "bleeding edges" "diamond rain" "ignite"	Songs visible in listbox	Songs listed properly	Pass
TC04	Play playlist sequentially	Multiple songs	Display songs in order of addition	Displayed in order	Pass
TC05	Shuffle playlist	Multiple songs	Songs order should change randomly	Songs shuffled	Pass
TC06	Remove song from an empty playlist	Empty playlist	Warning message	Warning shown	Pass

Table 3: test case summary

3.2 demonstration presentation slide and survey report attached along with the assignment.

4. Assess the effectiveness of data structures and algorithms

4.1 Understanding Asymptotic Analysis and Algorithm Efficiency

In the field of computer science, particularly in data structures and algorithms (DSA), evaluating the performance of algorithms is crucial. Asymptotic analysis serves as a fundamental tool in this evaluation, providing insights into an algorithm's efficiency as input sizes grow. This report delves into the concept of asymptotic analysis, its role in assessing algorithm effectiveness, and the primary metrics used to measure algorithm efficiency (GeeksforGeeks, 2019).

4.1.1 Asymptotic Analysis: Definition and Purpose

Asymptotic analysis is a mathematical approach used to describe the behavior of algorithms as the input size approaches infinity. Rather than focusing on exact execution times, it emphasizes the growth rate of an algorithm's resource consumption (such as time or space) relative to the input size. This abstraction allows for the comparison of algorithms independently of hardware or implementation specifics (geeksforgeeks, 2024).

The primary notations used in asymptotic analysis include:

- **Big O (O):** Represents the upper bound, indicating the worst-case scenario.

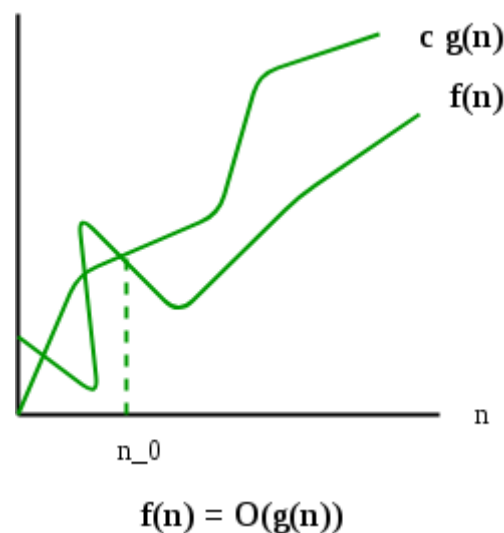


Figure 18: Big O (geeksforgeeks, 2024)

- **Big Omega (Ω):** Denotes the lower bound, reflecting the best-case scenario.

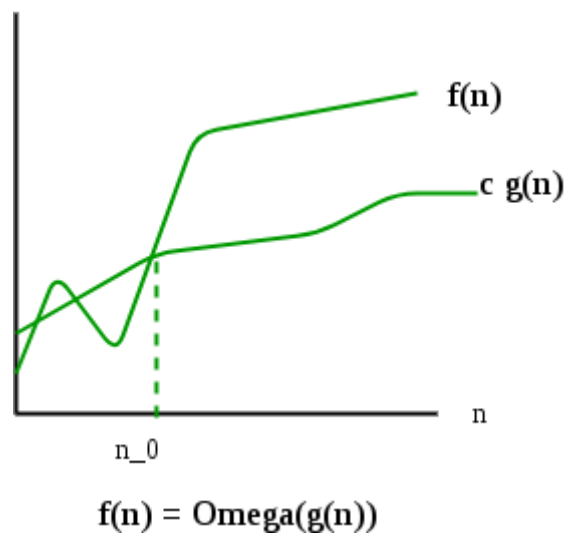


Figure 19: Omega (geeksforgeeks, 2024)

- **Big Theta (Θ):** Defines the tight bound, where the algorithm's performance is sandwiched between the best and worst cases.

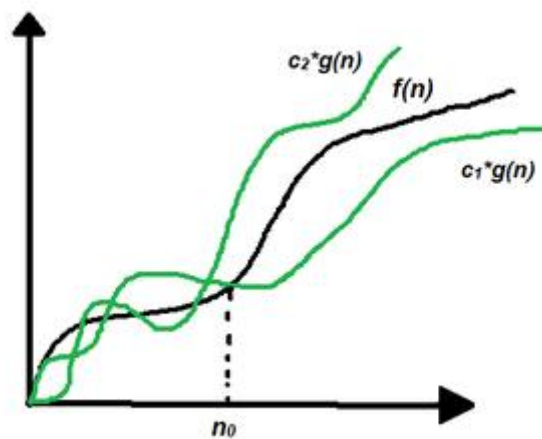


Figure 20: Theta (geeksforgeeks, 2024)

These notations provide a high-level understanding of an algorithm's efficiency, facilitating informed decisions during algorithm selection and optimization.

4.1.2 Assessing Algorithm Effectiveness Through Asymptotic Analysis

Asymptotic analysis aids in evaluating an algorithm's effectiveness by highlighting its scalability and performance trends. For instance, consider two sorting algorithms:

- **Algorithm A:** Exhibits a time complexity of $O(n^2)$.
- **Algorithm B:** Demonstrates a time complexity of $O(n \log n)$.

While both may perform adequately on small datasets, Algorithm B will significantly outperform Algorithm A as the dataset size increases. This insight is crucial when dealing with large-scale applications, ensuring that chosen algorithms remain efficient under growing workloads (Jeff, 2019).

Moreover, asymptotic analysis abstracts away constant factors and lower-order terms, focusing solely on the dominant growth rates. This abstraction simplifies the comparison between algorithms, emphasizing their fundamental efficiency characteristics.

4.1.3 Measuring Algorithm Efficiency: Two Key Metrics

Algorithm efficiency can be quantified using various metrics, with the two most prominent being:

a. Time Complexity

Time complexity measures the amount of computational time an algorithm requires relative to the input size. It provides an upper bound on the running time, facilitating predictions about performance and responsiveness. Common time complexities include:

- **$O(1)$** : Constant time.
- **$O(\log n)$** : Logarithmic time.
- **$O(n)$** : Linear time.
- **$O(n \log n)$** : Linearithmic time.
- **$O(n^2)$** : Quadratic time.

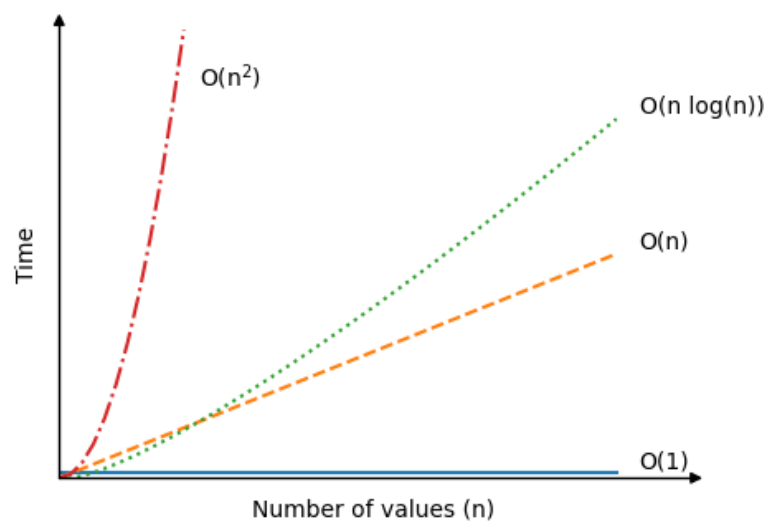


Figure 21: Time Complexity illustration (schools, 2025)

Understanding an algorithm's time complexity is vital for applications where execution speed is critical (geeksforgeeks, 2024).

b. Space Complexity

Space complexity assesses the amount of memory an algorithm consumes concerning the input size. This metric is essential when working with memory-constrained environments or large datasets. For example, an algorithm with $O(n)$ space complexity requires memory proportional to the input size, whereas an $O(1)$ algorithm uses a constant amount of memory regardless of input (geeksforgeeks, 2024).

Evaluating space complexity ensures that algorithms are not only fast but also memory-efficient, which is crucial for optimizing resource utilization.

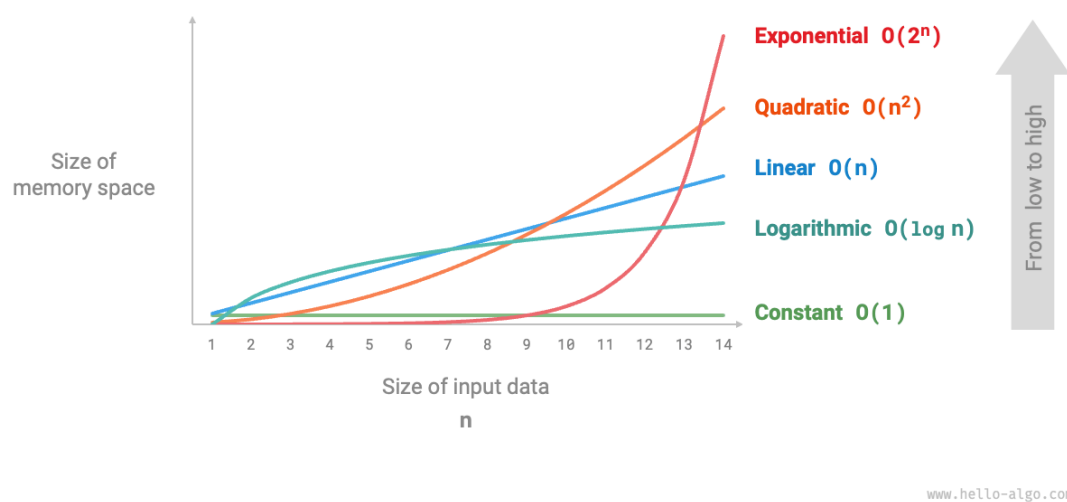


Figure 22: space complexity illustration (krahets, 2025)

Asymptotic analysis is a cornerstone in the study and application of algorithms, offering a framework to evaluate and compare algorithm efficiency objectively. By focusing on growth rates and abstracting away implementation details, it enables developers and computer scientists to make informed decisions, ensuring optimal performance and resource utilization in software applications.

4.2 Comparative Analysis of Array, Linked List, and Stack ADTs

In computer science, Abstract Data Types (ADTs) such as arrays, linked lists, and stacks are fundamental structures that dictate how data is organized and manipulated. Each ADT offers unique advantages and trade-offs concerning development time, implementation complexity, runtime efficiency, and accuracy.

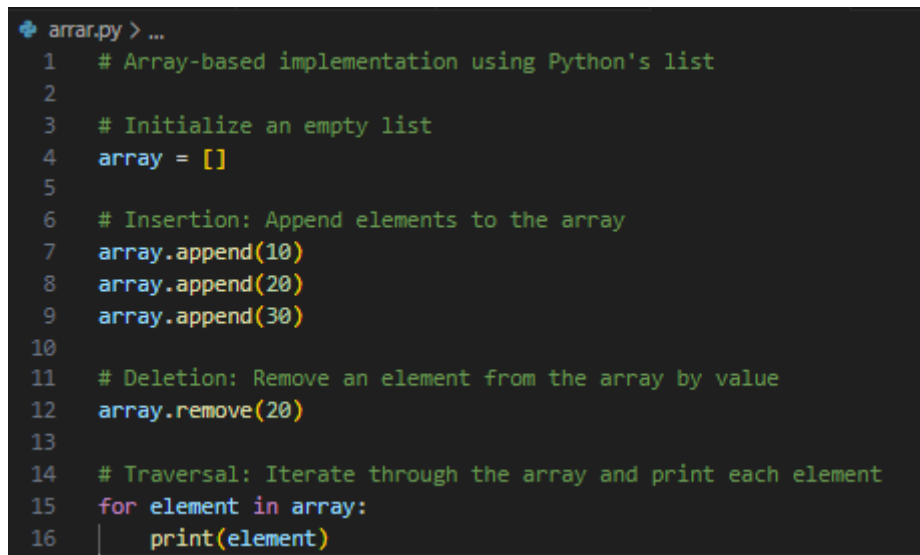
4.2.1 Program Implementations

a. Array-Based Program

Description: This program utilizes an array to store and manage a collection of integers. It includes functionalities to insert, delete, and search for elements.

Key Operations:

- **Insertion:** Adding an element at a specific index.
- **Deletion:** Removing an element from a specific index.
- **Search:** Finding the index of a particular element.



```
array.py > ...
1  # Array-based implementation using Python's list
2
3  # Initialize an empty list
4  array = []
5
6  # Insertion: Append elements to the array
7  array.append(10)
8  array.append(20)
9  array.append(30)
10
11 # Deletion: Remove an element from the array by value
12 array.remove(20)
13
14 # Traversal: Iterate through the array and print each element
15 for element in array:
16     print(element)
```

Figure 23: creating a program containing array as main ADT

b. Linked List-Based Program

Description: This program employs a singly linked list to manage a sequence of integers. It supports dynamic insertion and deletion of nodes.

Key Operations:

- **Insertion:** Adding a node at the beginning, end, or a specific position.
- **Deletion:** Removing a node from a specific position.
- **Traversal:** Iterating through the list to display elements.

```
LinkedList.py > ...
1 class Node:
2     """Class to represent a node in the linked list."""
3     def __init__(self, data):
4         self.data = data
5         self.next = None
6
7 class LinkedList:
8     """Class to represent the linked list."""
9     def __init__(self):
10         self.head = None
11
12     def insert_at_beginning(self, new_data):
13         """Insert a new node at the beginning of the list."""
14         new_node = Node(new_data)
15         new_node.next = self.head
16         self.head = new_node
17
18     def delete_node(self, key):
19         """Delete the first occurrence of a node with the given key."""
20         temp = self.head
21
22         # If the head node itself holds the key
23         if temp is not None:
24             if temp.data == key:
25                 self.head = temp.next
26                 temp = None
27                 return
28
29         # Search for the key to be deleted
30         prev = None
31         while temp is not None:
32             if temp.data == key:
33                 break
34             prev = temp
35             temp = temp.next
36
37         # Key was not present in the list
38         if temp == None:
39             return
40
41         # Unlink the node from the linked list
42         prev.next = temp.next
43         temp = None
44
45     def traverse(self):
46         """Print the linked list."""
47         temp = self.head
48         while temp:
49             print(temp.data)
50             temp = temp.next
51
52     # Example usage:
53     llist = LinkedList()
54     llist.insert_at_beginning(10)
55     llist.insert_at_beginning(20)
56     llist.insert_at_beginning(30)
57
58     llist.traverse()
59     # Output: 30, 20, 10
60
61     llist.delete_node(20)
62     llist.traverse()
63     # Output: 30, 10
64
```

Figure 24: creating a program containing linked list as a main ADT

c. Stack-Based Program

Description: This program implements a stack using either an array or a linked list, adhering to the Last-In-First-Out (LIFO) principle. It includes push and pop operations.

Key Operations:

- **Push:** Adding an element to the top of the stack.
- **Pop:** Removing the top element from the stack.
- **Peek:** Viewing the top element without removing it.

```
stack-based.py > ...
1  class Stack:
2      """Class to represent a stack using a list."""
3      def __init__(self):
4          self.stack = []
5
6      def push(self, item):
7          """Add an item to the top of the stack."""
8          self.stack.append(item)
9
10     def pop(self):
11         """Remove and return the top item of the stack."""
12         if not self.is_empty():
13             return self.stack.pop()
14         else:
15             return None
16
17     def peek(self):
18         """Return the top item of the stack without removing it."""
19         if not self.is_empty():
20             return self.stack[-1]
21         else:
22             return None
23
24     def is_empty(self):
25         """Check if the stack is empty."""
26         return len(self.stack) == 0
27
28     def size(self):
29         """Return the number of items in the stack."""
30         return len(self.stack)
31
32     # Example usage:
33     stack = Stack()
34     stack.push(10)
35     stack.push(20)
36     stack.push(30)
37
38     print(stack.peek()) # Output: 30
39     print(stack.pop())  # Output: 30
40     print(stack.size()) # Output: 2
```

Figure 25: creating a program containing stack as a main ADT

4.2.2 Trade-Off Analysis

The selection of an appropriate ADT involves evaluating trade-offs across various factors:

a. Development Time

- **Array:** Arrays are straightforward to implement due to their fixed size and direct indexing, resulting in shorter development time.
- **Linked List:** Implementing linked lists requires careful handling of pointers, increasing development complexity and time.
- **Stack:** Stacks built upon arrays or linked lists inherit the development complexities of their underlying structures (Rao, 2023).

b. Implementation Complexity

- **Array:** Fixed-size arrays are simple but lack flexibility. Dynamic arrays add complexity due to resizing mechanisms.
- **Linked List:** Managing dynamic memory allocation and pointer references adds to implementation complexity (QuickCodingExplanation, 2024).
- **Stack:** Stacks offer a simplified interface (push/pop), reducing complexity, especially when built upon arrays (Shreyans, 2015).

c. Runtime Efficiency

- **Array:** Provides constant-time access ($O(1)$) but insertion and deletion operations can be costly ($O(n)$) due to shifting elements.
- **Linked List:** Insertion and deletion are efficient ($O(1)$) when performed at the head, but accessing elements is slower ($O(n)$) due to sequential traversal.
- **Stack:** Both push and pop operations are efficient ($O(1)$), making stacks suitable for scenarios requiring rapid data access and modification.

d. Accuracy

- **Array:** Fixed size can lead to overflow errors if not managed properly. Dynamic arrays mitigate this but introduce complexity.
- **Linked List:** Dynamic sizing reduces overflow risks, but pointer errors can lead to data corruption.
- **Stack:** Stacks provide controlled access, reducing the likelihood of errors, especially when used with proper bounds checking.

4.3.3 Understanding Trade-Offs in ADT Selection

A trade-off in specifying an ADT involves balancing the benefits and drawbacks of different data structures to meet specific application requirements.

Example: Consider implementing a text editor's undo functionality.

- **Using a Stack:** Each action is pushed onto a stack, allowing for efficient undo operations (LIFO).
- **Trade-Off:** While stacks provide efficient undo capabilities, they may not efficiently support redo operations or random access to past actions.

This example illustrates the importance of evaluating the specific needs of an application and selecting an ADT that offers the most appropriate balance of performance and functionality.

Conclusion

Throughout this assignment, the importance of abstract data types (ADTs) in object-oriented software design has been clearly established. ADTs like Arrays, Linked Lists, and Stacks offer structured methods for data storage and manipulation, each with its own strengths and compromises depending on the problem domain.

By implementing and comparing these ADTs, it became evident that no single data structure is universally optimal. The choice between ADTs depends on factors like development and implementation time, runtime efficiency, accuracy, and adaptability to changing requirements. Furthermore, the use of asymptotic analysis equips developers with a theoretical foundation for evaluating the performance and scalability of algorithms. Ultimately, the thoughtful application of ADTs leads to more robust, efficient, and maintainable software systems, reaffirming their central role in both academic studies and real-world software development.

References

- Alex, 2023. *Learncpp.com*. [Online]
Available at: <https://www.learncpp.com/cpp-tutorial/the-benefits-of-data-hiding-encapsulation/>
[Accessed 12 03 2025].
- Editorial, H., 2025. *Hero Vired*. [Online]
Available at: <https://herovired.com/learning-hub/blogs/4-pillars-of-oop/>
[Accessed 22 3 2025].
- EnableGeek, 2024. *www.enablegeek.com*. [Online]
Available at: <https://www.enablegeek.com/blog/encapsulation-and-information-hiding-in-object-oriented-programming/>
[Accessed 13 3 2025].
- GeeksforGeeks, 2014. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/data-structures/>
[Accessed 10 3 2025].
- geeksforgeeks, 2015. *geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/queue-data-structure/>
[Accessed 17 3 2025].
- GeeksforGeeks, 2019. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/comparison-among-bubble-sort-selection-sort-and-insertion-sort>
[Accessed 1 4 2025].
- GeeksforGeeks, 2019. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/comparison-among-bubble-sort-selection-sort-and-insertion-sort/>
[Accessed 2 4 2025].
- geeksforgeeks, 2024. *geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/types-of-asymptotic-notations-in-complexity-analysis-of-algorithms>
[Accessed 2 4 2025].
- geeksforgeeks, 2025. *geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/binary-tree-data-structure/>
[Accessed 17 3 2025].
- geeksforgeeks, 2025. *geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>
[Accessed 17 3 2025].
- Geeksforgeeks, 2025. *Geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/stack-data-structure/>
[Accessed 17 3 2025].
- Geeksforgeeks, 2025. *Geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/tree-data-structure/>
[Accessed 17 3 2025].
- GeeksforGeeks, 2025. *GeeksforGeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>
[Accessed 10 3 2025].
- Jeff, E., 2019. *Algorithms*, s.l.: s.n.
- Klukowska, J., n.d. *CSCI-UA 102 Lecture 5: ADT: Lists, Stacks, Queues*, s.l.: cs.nyu.edu.
- krahets, 2025. *Hello-algo.com*. [Online]
Available at: https://www.hello-algo.com/en/chapter_computational_complexity/space_complexity/#241-space-related-to-algorithms
[Accessed 3 4 2025].
- programiz, n.d. *www.programiz.com*. [Online]
Available at: <https://www.programiz.com/dsa/stack>
[Accessed 10 3 2025].
- programiz, n.d. *www.programiz.com*. [Online]
Available at: <https://www.programiz.com/dsa/queue>
[Accessed 10 3 2025].

pythonds, n.d. *runestone.academy*. [Online]
 Available at: <https://runestone.academy/ns/books/published/pythonds/BasicDS/TheStackAbstractDataType.html>
 [Accessed 12 3 2025].

QuickCodingExplanation, 2024. *medium*. [Online]
 Available at: <https://quickcodingexplanation.medium.com/data-structures-overview-arrays-stack-queue-linked-list-hash-table-heap-binary-tree-7b88a5711a0b>
 [Accessed 3 4 2025].

Rao, 2023. *CSE 373 Lecture 5: Lists, Stacks, and Queues* ♦ *We will review*, Washinton: s.n. schools, W., 2025. *W3 schools.com*. [Online]
 Available at: https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php
 [Accessed 3 4 2025].

Shreyans, 2015. *Stack Overflow*. [Online]
 Available at: <https://stackoverflow.com/questions/33780618/stack-adt-abstract-data-type-implementation-array-vs-linked>
 [Accessed 3 4 2025].

S, R. A., 2025. *simplilearn.com*. [Online]
 Available at: <https://www.simplilearn.com/tutorials/data-structure-tutorial/graphs-in-data-structure>
 [Accessed 17 3 2025].

Team, K. C., 2024. *Kodezi Blog*. [Online]
 Available at: <https://blog.kodezi.com/understanding-the-4-concepts-of-oop-a-complete-tutorial/>
 [Accessed 22 3 2025].

Thomas H Cormen. Charles Eric Leiserson, R. L. R. C. S., 2009. *Introduction to Algorithms*. 3rd ed. s.l.:MIT Press.

user1849655, 2012. *Stack Overflow*. [Online]
 Available at: <https://stackoverflow.com/questions/14068767/why-stack-is-called-abstract-data-type-if-stack-is-adt-then-how-to-implement-s>
 [Accessed 12 3 2025].